

Chapter 6. Data Processing

In the trivial recommender that we defined in [Chapter 1](#), we used the method `get_availability`; and in the MPIR, we used the method `get_item_popularities`. We hoped the choice of naming would provide sufficient context about their function, but we did not focus on the implementation details. Now we will start unpacking the details of some of this complexity and present the toolsets for online and offline collectors.

Hydrating Your System

Getting data into the pipeline is punnily referred to as *hydration*. The ML and data fields have a lot of water-themed naming conventions; [“\(Data ∩ Water\) Terms”](#) by Pardis Noorzad covers this topic.

PySpark

Spark is an extremely general computing library, with APIs for Java, Python, SQL, and Scala. PySpark’s role in many ML pipelines is for data processing and transforming the large-scale datasets.

Let’s return to the data structure we introduced for our recommendation problem; recall that the user-item matrix is the linear-algebraic representation of all the triples of users, items, and the user’s rating of the item. These triples are not naturally occurring in the wild. Most commonly, you begin with log files from your system; for example, Bookshop.org may have something that looks like this:

```
'page_view_id': 'd15220a8e9a8e488162af3120b4396a9ca1',  
'anonymous_id': 'e455d516-3c08-4b6f-ab12-77f930e2661f',  
'view_tstamp': 2020-10-29 17:44:41+00:00,
```

```
'page_url': 'https://bookshop.org/lists/best-sellers-of-the-week',
'page_url_host': 'bookshop.org',
'page_url_path': '/lists/bookshop-org-best-sellers-of-the-week',
'page_title': 'Best Sellers of the Week',
'page_url_query': None,
'authenticated_user_id': 15822493.0,
'url_report_id': 511629659.0,
'is_profile_page': False,
'product_viewed': 'list',
```

This is a made-up log file that may look similar to the backend data for Bookshop.org’s best sellers of the week. These are the kinds of events that you consume from engineering and are likely stored in your columnar database. For data like this, utilizing SQL syntax will be our entry point.

PySpark provides a convenient SQL API. Based on your infrastructure, this API will allow you to write what looks like SQL queries against a potentially massive dataset.

EXAMPLE SCHEMAS

These example database schemas are only guesses at what Bookshop.org may use, but they are modeled on the authors’ experience of looking at hundreds of database schemas at multiple companies over many years. Additionally, we attempt to distill these schemas to the components relevant to our topic. In real systems, you’d expect much more complexity but the same essential parts. Each data warehouse and event stream will have its own quirks. Please consult a data engineer near you.

Let’s use Spark to query the preceding logs:

```
user_item_view_counts_qry = """
SELECT
    page_views.authenticated_user_id
    , page_views.page_url_path
    , COUNT(DISTINCT page_views.page_view_id) AS count_views

FROM prod.page_views
JOIN prod.dim_users
```

```

        ON page_views.authenticated_user_id = dim_users.authenticated_user_id

WHERE DATE page_views.view_tstamp >= '2017-01-01'
      AND dim_users.country_code = 'US'

GROUP BY
    page_views.authenticated_user_id
    , page_views.page_url_path

ORDER BY 3, page_views.authenticated_user_id
      """

user_item_view_counts_sdf = spark.sql(user_item_view_counts_qry)

```

This is a simple SQL query, assuming the preceding log schema, that would allow us to see, for each user-item pair, how many times that user has viewed that pair. The convenience of writing pure SQL here means that we can use our experience in columnar databases to quickly ramp up on Spark.

The major advantage of Spark, however, is not yet on display. When executing the preceding code in a Spark session, this query will not be immediately run. It will be staged for execution, but Spark waits until you use this data downstream in a way that *requires immediate execution* before it begins doing so. This is called *lazy evaluation*, and it allows you to work on your data object without every change and interaction immediately being applied. For more details, it's worth consulting a more in-depth guide like [Learning Spark](#) by Jules Damji et al. (O'Reilly), but there's one more important characteristic of the Spark paradigm that is essential to discuss.

Spark is natively a distributed computing language. In particular, this means that the preceding query—even after we force it to execute—will store its data on multiple computers. Spark works via a *driver program* in your program or notebook, which drives a *cluster manager*, which in turn coordinates *executors* on *worker nodes*. When we query data with Spark, instead of all that data being returned into a DataFrame in memory on the computer we're using, parts of that data are sent to memory on the executors. And when we do a transformation on the DataFrame, it is ap-

plied appropriately on the pieces of the DataFrame that are stored on each of the executors.

If this sounds a bit like magic, that's because it's obscuring a lot of technical details behind several convenience layers. Spark is a layer of technology that allows the ML engineer to program as if they're working on one machine, and have those changes take effect on an entire cluster of machines. It's not important to understand the network structure when querying, but it is important to be aware of some of these details in case things go wrong; the ability to understand what the error output is referring to is crucial in troubleshooting. This is all summarized in [Figure 6-1](#), which is a diagram from the [Spark documentation](#).

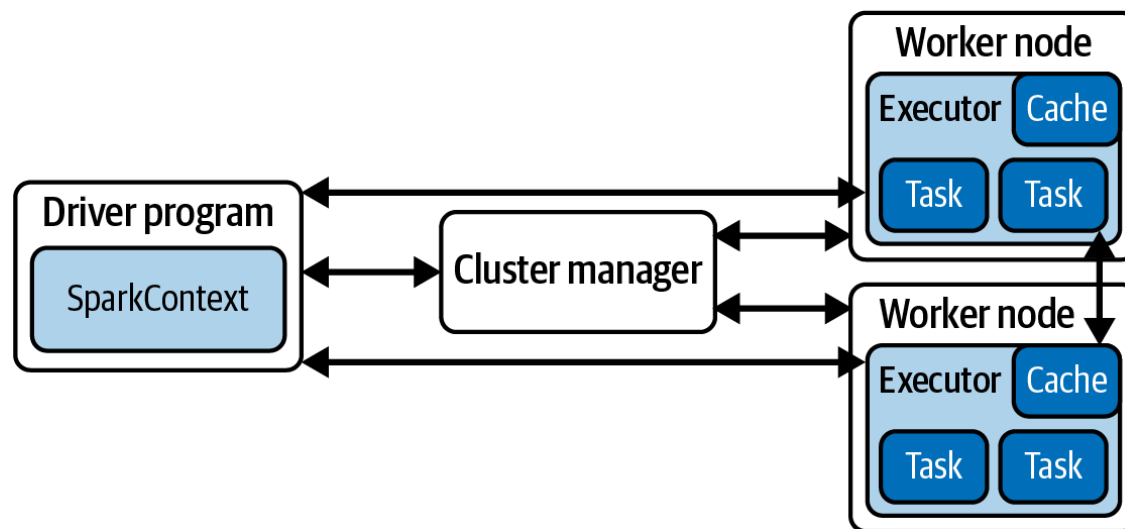


Figure 6-1. Component architecture of Spark 3.0

It's important to note that all this does not come for free; both lazy evaluation and distributed DataFrames come at the cost of needing additional thought when writing programs. Even though Spark makes a lot of this work far easier, understanding how to write efficient code in this paradigm that works with the architecture but still achieves complicated goals can require a year's worth of experience.

Returning to recommendation systems—and in particular, the offline collector—we want to use PySpark to build the types of datasets needed to train our models. One simple thing to do with PySpark is to transform our logs data into the appropriate form for training a model. In our simple query, we applied a few filters to our data and grouped by user and item

to get the number of views. A variety of other tasks may fit naturally into this paradigm—perhaps adding user or item features stored in other databases, or high-level aggregations.

In our MPIR, we asked for `get_item_popularities`; and we sort of assumed a few things:

- This would return the number of times each item was chosen.
- This method would be fast.

The second point is important if the endpoint is going to be called in real time. So how might Spark come into play?

First, let's assume we have a lot of data, enough that we can't get it all to fit into our little MacBook Pro's memory. Additionally, let's continue to use the preceding schema. We can write an even simpler query:

```
item_popularity_qry = """
SELECT
    page_views.page_url_path
    , COUNT(DISTINCT page_views.authenticated_user_id) AS count_viewers

FROM prod.page_views
JOIN prod.dim_users
    ON page_views.authenticated_user_id = dim_users.authenticated_user_id

WHERE DATE page_views.view_tstamp >= '2017-01-01'
    AND dim_users.country_code = 'US'

GROUP BY
    page_views.page_url_path

ORDER BY 2
"""

item_view_counts_sdf = spark.sql(item_popularity_qry)
```

We can now write this aggregated list of `(item, count)` pairs to an app database to serve `get_item_popularities` (something that doesn't require us to do any parsing when this is called), or potentially we can take

a subset of the top- N of this list and store it in memory to get the best items with respect to a particular ranking. Either way, we've separated concerns of parsing all our log data, and doing aggregation, from the `get_item_popularities` function call in real time.

This example used an overly simple data aggregation, one just as easy to do in something like PostgreSQL, so why bother? The first reason is scalability. Spark is really built to scale horizontally, which means that as the data we need to access grows, we merely add more worker nodes.

The second reason is that PySpark is more than just SparkSQL; anyone who's done complicated SQL queries can probably agree that the power and flexibility of SQL is enormous, but frequently certain tasks that you want to achieve require a lot of creativity to carry out in the fully SQL environment. PySpark gives you all the expressiveness of pandas DataFrames, Python functions and classes, and a simple interface to apply Python code to the PySpark data structure's user-defined functions (UDFs). UDFs are similar to lambda functions that you'd use in pandas, but they're built and optimized for PySpark DataFrames. As you've probably experienced when writing ML programs in smaller data regimes, at some point you switch away from using only SQL to using pandas API functions to perform data transformations—so too will you appreciate this power at the Spark data scale.

PySpark allows you to write what looks very much like Python and pandas code and have that code executed in a distributed fashion! You don't need to write code to specify which worker nodes operations should happen; that's handled for you by PySpark. This framework isn't perfect; some things you expect to work may require a bit of care, and optimization of your code can require an additional level of abstraction, but generally, PySpark gives you a rapid way to move your code from one node to a cluster and utilize that power.

To illustrate something a bit more useful in PySpark, let's return to collaborative filtering (CF) and compute some features more relevant for ranking.

Example: User Similarity in PySpark

A user similarity table allows you to map a user to other users who are relevant to the recommender. This recalls the assumption that two similar users like similar things, and thus you can recommend to both users the items that one hasn't seen. Constructing this user similarity table is an example of a PySpark job that you might see as part of the offline collector's responsibility. Even though in many cases ratings would continue to stream in all the time, for the purposes of large offline jobs, we often think of a daily batch to update the essential tables for our model. In practice, in many cases this daily batch job suffices to provide features that are good enough for most of the ML work downstream. Other important paradigms exist, but those frequently *marry* the more frequent updates with these daily batch jobs, instead of totally eliminating them.

This architecture of daily batch jobs with smaller, more frequent batch jobs is called the *lambda architecture*, and we'll get more into the details of how and why later. In brief, the two layers—batch and speed—which are distinguished (inversely) by the frequency of processing and the volume per run of data they process. Note that the speed layer may have varying frequencies associated with it, and it's possible to have different speed layers for hourly, and another speed layer for minute-frequency jobs that do different things. [Figure 6-2](#) provides an overview of the architecture.

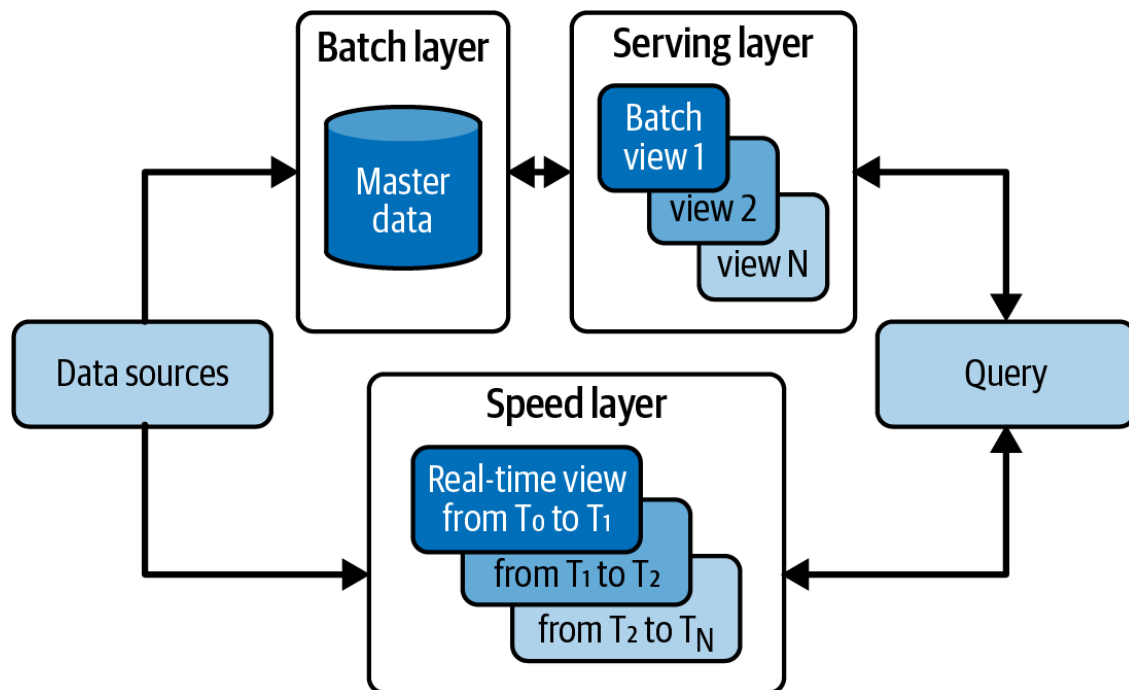


Figure 6-2. Overview of a lambda architecture

In the case of user similarity, let's work on a batch job implementation of computing a daily table. First we'll need to get ratings from our schema before today. We'll also include a few other filters that simulate how this query might look in real life:

```
user_item_ratings_qry = """
SELECT
    book_ratings.book_id
    book_ratings.user_id
    , book_ratings.rating_value
    , book_ratings.rating_tstamp

FROM prod.book_ratings
JOIN prod.dim_users
    ON book_ratings.user_id = dim_users.user_id
JOIN prod.dim_books
    ON book_ratings.book_id = dim_books.dim_books

WHERE
    DATE book_ratings.rating_tstamp
        BETWEEN (DATE '2017-01-01')
        AND (CAST(current_timestamp() as DATE)
    AND book_ratings.rating_value IS NOT NULL
    AND dim_users.country_code = 'US'
    AND dim_books.book_active
```



```
user_item_ratings_sdf = spark.sql(user_item_ratings_qry)
```

As before, utilizing the SQL syntax to get the dataset into a Spark DataFrame is the first step, but now we have additional work on the PySpark side. A common pattern is to get the dataset you want to work with via simple SQL syntax and logic, and then use the PySpark API to do more detailed data processing.

Let's first observe that we have no assumptions about uniqueness of a user-item rating. For the sake of this table, let's decide that we'll use the most recent rating for a pair:

```
from pyspark.sql.window import Window

windows = Window().partitionBy(
    ['book_id', 'user_id']
).orderBy(
    col("rating_tstamp").desc()
)

user_item_ratings_sdf.withColumn(
    "current_rating",
    first(
        user_item_ratings_sdf("rating_tstamp")
    ).over(windows).as("max_rating_tstamp")
).filter("rating_tstamp = max_rating_tstamp")
```

We'll now use `current_rating` as our ratings column for the purpose of downstream calculation. Recall from before our ratings-based definition of user similarity:

$$\text{USim}_{A,B} = \frac{\sum_{x \in \mathcal{R}_{A,B}} (r_{A,x} - \bar{r}_A) (r_{B,x} - \bar{r}_B)}{\sqrt{\sum_{x \in \mathcal{R}_{A,B}} (r_{A,x} - \bar{r}_A)^2} \sqrt{\sum_{x \in \mathcal{R}_{A,B}} (r_{B,x} - \bar{r}_B)^2}}$$

The important values we'll need are as follows:

$$r(-,-)$$

The rating corresponding to a user-item pair

$\bar{r}_{(-)}$

The average rating across all items for a user

The rows are already the $r_{(-,-)}$ values, so let's compute user average ratings, $\bar{r}_{(-)}$, and the rating deviations:

```
from pyspark.sql.window import Window
from pyspark.sql import functions as F

user_partition = Window.partitionBy('user_id')

user_item_ratings_sdf = user_item_ratings_sdf.withColumn(
    "user_average_rating",
    F.avg("current_rating").over(user_partition)
)

user_item_ratings_sdf = user_item_ratings_sdf.withColumn(
    "rating_deviation_from_user_mean",
    F.col("current_rating") - F.col("user_average_rating")
)
```

Now our schema should look like this (we've formatted it slightly nicer than the default Spark output):

```
+-----+-----+-----+-----+
|book_id|user_id|rating_value|rating_tstamp|
+-----+-----+-----+-----+
+-----+-----+-----+-----+
|current_rating|user_average_rating|rating_deviation_from_user_mean|
+-----+-----+-----+-----+
```

Let's finish creating a dataset that contains our User Similarity calculations:

```
user_pair_item_rating_deviations = user_item_ratings_sdf.alias("left_ratings")
    .join(user_item_ratings_sdf.alias("right_ratings"),
    (
```

```

F.col("left_ratings.book_id") == F.col("right_ratings.book_id") &\
F.col("left_ratings.user_id") != F.col("right_ratings.user_id")
    ),
    "inner"
).select(
    F.col("left_ratings.book_id"),
    F.col("left_ratings.user_id").alias("user_id_1"),
    F.col("right_ratings.user_id").alias("user_id_2"),
    F.col("left_ratings.rating_deviation_from_user_mean").alias("dev_1"),
    F.col("right_ratings.rating_deviation_from_user_mean").alias("dev_2")
).withColumn(
    'dev_product',
    F.col("dev_1")*F.col("dev_2")
)

user_similarities_sdf = user_pair_item_rating_deviations.groupBy(
    "user_id_1", "user_id_2"
).agg(
    sum('dev_product').alias("dev_product_sum"),
    sum(F.pow(F.col("dev_1"),2)).alias("sum_of_sqrd_devs_1"),
    sum(F.pow(F.col("dev_2"),2)).alias("sum_of_sqrd_devs_2")
).withColumn(
    "user_similarity",
    (
        F.col("dev_product_sum") / (
            F.sqrt(F.col("sum_of_sqrd_devs_1")) *
            F.sqrt(F.col("sum_of_sqrd_devs_2"))
        )
    )
)

```

In constructing this dataset, we begin by taking a self-join, which avoids matching the same users with themselves but rather joins on books that match. As we do this join, we take the rating deviation from the user's mean ratings that we computed previously. We also use this opportunity to multiply them together for the numerator in our user similarity function. In the last step, we'll `groupBy` again so that we can sum over all matching book IDs (by `groupBy` on `user_id_1` and `user_id_2`); we sum the product and the powers of each set of deviations so that we can finally divide and generate a new column for our user similarity.

While this computation isn't particularly complex, let's take note of a few things that we might appreciate. First, we built our user similarity matrix in full from our records. This matrix may now be stored in a faster-access format so that if we wish to do operations in real time, it's ready to go. Second, we did all these data transformations in Spark, so we can run these operations on massive datasets and let Spark handle the parallelization onto the cluster. We even were able to do this while writing code that looks a lot like pandas and SQL. Finally, all our operations were columnar and required no iteration-based calculation. This means this code will scale much better than some approaches. This also ensures that Spark can parallelize our code well, and we can expect good performance.

We've seen how PySpark can be used to prepare our user similarity matrix. We have this definition of affinity estimating the appropriateness of an item for a user; we can collect each of those scores into a tabular form—user rows and item columns—to yield a matrix. As an exercise, can you take this matrix and generate the affinity matrix?

$$\text{Aff}_{A,i} = \bar{r}_A + \frac{\sum_{U \in \mathcal{N}(A)} \text{USim}_{A,U}^*(r_{U,i} - \bar{r}_A)}{\sum_{U \in \mathcal{N}(A)} \text{USim}_{A,U}}$$

Feel free to assume that $\mathcal{N}(A)$ is just the five nearest neighbors to A with respect to user similarity.

DataLoaders

DataLoaders is a programming paradigm originating from PyTorch, but it has been embraced in other gradient-optimized ML workflows. As we begin to integrate gradient-based learning into our recommendation system architectures, we will face challenges in our MLOps tooling. The first is related to training data size and available memory. DataLoaders are a way to prescribe how data is batched and sent to the training loop efficiently; as datasets get large, careful scheduling of these training sets can have major effects on learning. But why must we think about *batches* of data? That's because we'll use a variant of gradient descent appropriate for large amounts of data.

First, let's review the basics of *mini-batched gradient descent*. During training via gradient descent, we make a forward pass of our training sample through our model to yield a prediction, and we then compute the error and the appropriate gradient backward through our model to update parameters. Batched gradient descent takes all our data in a single pass to compute the gradient for the training set and push it back through; this implies you have the entire training dataset in memory. As the dataset scales, this ranges from expensive to impossible; to avoid this, we can instead compute gradients of the loss function for only a subset of the dataset at a time. The simplest paradigm for this, called *stochastic gradient descent* (SGD), computes these gradients and parameter updates one sample at a time. The mini-batched version performs our batched gradient descent, but over a series of subsets to form a partition of our dataset. In mathematical notation, we write the update rule in terms of Jacobians on the smaller batches:

$$\theta = \theta - \eta \nabla_{\theta} J(\theta; x^{(i:i+n)}, y^{(i:i+n)})$$

This optimization serves a few purposes. First, it requires only potentially small subsets of our data held in memory during the steps. Second, it requires far fewer passes than the purely iterative version in SGD. Third, the gradient operating on these mini-batches can be organized as a Jacobian, and thus we have linear-algebraic operations that may be highly optimized.

The mathematical notion of a Jacobian in the simplest sense is an organizational tool for a set of vector derivatives with relevant indexes. You may recall that for functions of several variables, you can take the derivative *with respect to* each of those variables. For a single multivariable scalar function, the Jacobian is simply the row vector of first derivatives of the function—which happens to be the transpose of the gradient.

This is the simplest case; the gradient of a multivariable scalar function may be written as a Jacobian. However, once we have a vector of (vector) derivatives, we can write that as a matrix; the utility here is really only in the notation, though. When you collect a series of multivariable scalar functions into a vector of functions, the associated vector of gradients is a vector of vectors of derivatives. This is called a *Jacobian matrix*, and it generalizes the gradient to vector-valued functions. As you’ve likely realized, layers of neural networks are a great source of vector-valued functions for which you’d like to derivate.

If you’re convinced mini-batches are useful, it’s time to discuss *DataLoaders*—a simple PyTorch API for facilitating mini-batch access from a large dataset. The key parameters for a `DataLoader` are `batch_size`, `shuffle`, and `num_workers`. The batch size is easy to understand: it’s the number of samples included in each batch (often an integer factor of the total size of the dataset). Often a shuffle operation is applied in serving up these batches; the shuffle allows batches in each epoch to be shown to the network in a randomized order; this is intended to improve robustness. Finally, `num_workers` is a parallelization parameter for the CPU’s batch generation.

The utility of a `DataLoader` is really best understood via demonstration:

```
params = {
    'batch_size': _,
    'shuffle': _,
    'num_workers': _
}

training_generator = torch.utils.data.DataLoader(training_set, params)
```

```

validation_generator = torch.utils.data.DataLoader(validation_set, params)

// Loop over epochs
for epoch in range(max_epochs):
    // Training
    for local_batch, local_labels in training_generator:

        // Model computations
        [...]

    // Validation
    with torch.set_grad_enabled(False):
        for local_batch, local_labels in validation_generator:

            // Model computations
            [...]

```

The first important detail in this code is that any of its generators will be reading in mini-batches from your total dataset and can be instructed to load those batches in parallel. Note also that any differential steps in the model computations will now be operating on these mini-batches.

It's easy to think of DataLoaders as merely a tool for code cleanliness (which, admittedly, it does improve), but it's important to not underestimate how the control of batch order, parallelization, and shape are significant features for training your model. Lastly, the structure of your code now looks like batch gradient descent, but it is taking advantage of mini-batching, further exposing what your code actually does instead of the steps necessary to do it.

Database Snapshots

Let's round out this section by stepping back from these fancy technologies to discuss something important and classic: snapshotting a production database.

An extremely likely scenario is that the engineers (potentially also you) who have built the recommendations server are writing their logs and other application data to an SQL database. More likely than not, this data-

base architecture and deployment are optimized for fast querying by the application across its most common use cases. As we've discussed, those logs may be in an event-style schema, and there are other tables that may require aggregation and roll-up to make any sense. For example, a *current inventory* table may require knowledge of start-of-day inventory and then aggregate a list of purchase events.

All told, the production SQL database is usually a crucial component in the stack that's geared to specific use. As the downstream consumer of this data, you may find yourself wanting different schemas, wanting lots of access to this database, and performing serious operations on this data. The most common paradigm is *database snapshotting*. Snapshotting is a functionality provided by various flavors of SQL to performantly make a clone of a database. While this snapshotting may take form in a variety of ways, let's focus on a few that serve to simplify our systems and ensure they have the necessary data on hand:

- A daily table snapshot may be tied to an `as_of` field, or *the state of this table on this day*.
- A daily table snapshot may be limited by time to see *what records have been added today*.
- An event table snapshot may be used to feed a set of events into an event stream processor like Segment (note that you may also set up live event streams like Kafka).
- An hourly aggregated table can be used for status logging or monitoring.

In general, the paradigm is usually to operate on snapshots for downstream data processing. Many of the kinds of data processing we mentioned earlier—like computing user similarity—are operations that may require significant data reads. *It's important to not build ML applications that require extensive querying on the production database*, because doing so would likely decrease performance of the app and result in a slower user experience. This decrease will undermine the improvement made possible by your recommendations.

Once you've snapshotted the tables you're interested in, you can often find a collection of data pipelines useful to transform that data into even more specific tables in your *data warehouse* (where you should be doing most of your work anyway). Tools like Dagster, dbt, Apache Airflow, Argo, and Luigi are popular data-pipeline and workflow orchestration tools for extract, transform, load (ETL) operations.

Data Structures for Learning and Inference

This section introduces three important data structures that will enable our recommendation system to perform complex operations quickly. The goal of each structure is to sacrifice precision as little as possible, while speeding up access to the data in real time. As you'll see, these data structures form the backbone of the real-time inference pipeline and approximate what takes place in the batch pipeline as accurately as possible.

The three data structures are as follows:

- Vector search/ANN index
- Bloom filters for candidate filtering
- Feature stores

So far, we've discussed the necessary components for getting data flowing in your system. These help organize data to make it more accessible during the learning and inference processes. Also, we'll find some shortcuts to speed up inference during retrieval. Vector search will allow us to identify similar items at scale. Bloom filters will allow us to rapidly evaluate many criteria for excluding results. Feature stores will provide us with necessary data about users for recommendation inference.

Vector Search

We have discussed user similarity and item similarity in terms of understanding the relationships between those entities, but we haven't talked about any *acceleration structures* for these processes.

First let's discuss a bit of terminology;. If we think of a collection of vectors that represent entities with a similarity metric provided by a distance function, we refer to this as a *latent space*. The simple goal is to utilize our latent space and its associated similarity metric (or complementary distance metric) to be able to retrieve *similar* items quickly. In our previous examples with similarity, we talked about neighborhoods of users and how they can be utilized to build an affinity score between users and unseen items. But how do you find the neighborhood?

To understand this, recall that we defined neighborhoods of an element $\mathbf{x}$, written $\mathcal{N}(\mathbf{x})$, as the set of k elements in the latent space with the maximum similarity; or said differently, the set of j th order statistics for $j \leq k$ from the sample of item similarities to $\mathbf{x}$. These *k-nearest neighbors*, as they're often called, will be used as the set of elements considered similar to $\mathbf{x}$.

These vectors from CF yield a few other useful side effects:

- A simple recommender that randomly samples unseen items from a user neighborhood's liked items
- Predictions about features of a user, from known features of users in the neighborhood
- User segmentation via taste similarity

So how can we speed up these processes? One of the first significant improvements in this area came from inverted indices. Utilizing inverted indices is at its core carefully constructing a large hash between tokens of the query (for text-based search) and the candidates.

This approach is great for tokenizable entities like sentences or small-lexicon collections. Given the ability to look up items that share one or many tokens with the query, you can even use a general latent embedding to rank the candidate responses by similarity. This approach deserves extra consideration as you scale: it incurs a speed cost because it entails two steps, and because the similarity distribution may not be well correlated with the token similarity required to return many more candidates than we need.

Classic approaches to building a search system are based on large lookup tables and feel deterministic. As we move toward ANN lookup, we want to relax some of that strong deterministic behavior and introduce data structures that make assumptions to *prune* these large indices. Instead of building indices for only tokenizable components of your elements, you could precompute the k -d tree and use the indices as the index. The k -d tree would precompute the nearest neighbors in a batch process (which may be slow), to populate a top- k response for fast lookup. k -d trees are an efficient data structure for encoding the preceding neighborhoods but are notoriously slow to read from in higher dimensions. Using them instead to build inverted indices, though, can be a great improvement.

More recently, explicitly using vector databases with vector search is becoming much more possible and feasible. Elasticsearch has added this capability; [Faiss](#) is a Python library that helps you implement this functionality in your systems; [Pinecone](#) is a vector-database system explicitly targeting this goal; and [Weaviate](#) is a native vector-database architecture that allows you to layer the previous token-based inverted indices and vector similarity search.

Approximate Nearest Neighbors

What are this element's k -nearest neighbors? Incredibly, approximate nearest neighbors (ANN) can get very high accuracy compared to the actual nearest neighbors, and you get there faster with head-spinning speedups. You often are satisfied with approximate solutions to these problems.

One open source library that specializes in these approximations is [PyNNDescent](#), which uses clever speedups via both optimized implementation and careful mathematical tricks. With ANN, you are opened up to two strategies as discussed:

- The pre-index can be dramatically improved.
- On queries without a pre-indexing option, you can still expect good performance.

In practice, these similarity lookups are incredibly important for making your applications actually work. While we've mostly talked about recommendations for full known catalogs of items, we cannot assume this in other recommendation contexts. These include the following:

- Query-based recommendations (like search)
- Contextual recommendations
- Cold-starting new items

As we go, you will see more and more references to similarity in spaces and nearest neighbors; at each of those moments, think: "I know how to make this fast!"

Bloom Filters

Bloom filters are probabilistic data structures that allow us to test for set inclusion very efficiently but with a downside: set exclusion is deterministic, but set inclusion is probabilistic. *In practice, this means that asking the question "Is x in this set" never results in a false negative but may result in a false positive!* Note that this type-I error increases as the size of the bloom increases.

Via vector search, we have identified a large pool of potential recommendations for the user. From this pool, we need to do some immediate elimination. The most obvious type of high-level filtering that's essential is to remove those items that the *user has previously not shown interest in or has already purchased*. You've probably had the experience of being recommended the same item, over and over, and thinking, "I don't want this; stop showing me this." From the simple CF models we've introduced, you may now see why this could happen.

The system has identified a set of items via CF that you're more likely to pick. Without any outside influence, those computations will continue to return the same results, and you'll never escape those recommendations. As the system designer, you may start with a heuristic:

If the user has seen this item recommended three times and never clicked, let's not show it to them anymore.

This is a totally reasonable strategy to improve *freshness* (the idea of ensuring users see new item recommendations) in your recommendation system. While this is a simple strategy to improve your recommendations, how might you implement this at scale?

A bloom filter may be used by defining the sets in question with the following: “Has this user seen this item recommended three times and never clicked?” Bloom filters have a caveat that they’re additive only: once something is in the bloom, you can’t remove it. This is not a problem when observing a binary state like this heuristic.

Let’s construct a user-item ID to use as our hash in the bloom. Remember that the key feature of the bloom filter is to quickly determine whether the hashed item is in the bloom. When we observe a user-item pair that satisfies the preceding criteria, take that pair as an ID and hash it. Now, because that hashed pair can be easily reconstructed from a list of items for a user, we have a very fast way to filter.

Let’s discuss a few technical details on this topic. First, you might want to do a variety of kinds of filtering—maybe freshness is one, and another may be items the user has already bought, and a third could exclude items that have sold out.

Here it would be good to implement each of these filters independently; the first two can follow our user-item ID hashing as before, and the third one can be a hash only on item IDs.

Another consideration is populating the bloom filters. It’s best practice to build these blooms from a database during the offline batch jobs. On whatever schedule your batch training is run, rebuild your blooms from the records storage to ensure you’re keeping your blooms accurate. Remember that blooms don’t allow deletion, so in the previous example, if an item goes from sold out to restocked, your batch refresh of your blooms can pick up the availability again. In between batch retraining, adding to a bloom is also very performant, so you can continue to add to

the bloom as you observe more data that needs to be considered for the filtering in real time. Be sure these transactions are logged to a table, though! That logging will be important when you want to refresh.

Fun Aside: Bloom Filters as the Recommendation System

Bloom filters not only provide an effective way to eliminate some recommendations based on conditions for inclusion, but can also be used to do the recommending itself! In particular, [“An Item/User Representation for Recommender Systems Based on Bloom Filters”](#) by Manuel Pozo et al. shows that for high-dimensional feature sets with a lot of sparsity (as we discussed in [Chapter 3](#)), the type of hashing bloom filters do can help overcome some of the key challenges in defining good similarity functions!

Let’s observe that we can do two natural operations on sets via the bloom filter data structures. First, consider two sets A and B , and associate to them bloom filters $\mathcal{BF}_A$ and $\mathcal{BF}_B$. Then what’s the definition of $A \cap B$? Can we come up with a bloom filter for this intersection? Yep! Recall that our bloom filters are guaranteed to tell us when an element is not contained in the set, but if an element is in the set, the bloom filter can respond with only a certain probability. In this case, we’d simply look for elements that are *in* according to $\mathcal{BF}_A$ AND *in* according to $\mathcal{BF}_B$. Of course, the set of items returned as *in* each set is larger than the actual set (i.e., $A \subset \mathcal{BF}_A$), so the intersection will also be larger:

$$A \cap B \subset \mathcal{BF}_A \cap \mathcal{BF}_B$$

Note that you can compute the exact difference in cardinality via information about your choice of hashing functions. Also note that the equation is an abuse of notation by calling $\mathcal{BF}_A$ the set of things returned by the bloom filter corresponding to A .

Second, we also need to construct the union. This is similarly easy by considering elements that are *in* according to $\mathcal{BF}_A$ OR *in* according to $\mathcal{BF}_B$. And so, similarly:

$$A \cup B \subset \mathcal{BF}_A \cup \mathcal{BF}_B$$

Now, if we consider items X and Y as concatenated vectors of potentially many features, and hash those concatenated features, we are representing each of them as the bitwise vectors of our bloom. From before, we saw that the intersection of two blooms makes sense, and in fact is equivalent to the bitwise *AND* of their bloom representations. This means two items' feature similarities can be expressed by the bitwise *and* similarity of their bloom hashes:

$$\text{sim}(X, Y) = |\mathcal{BF}(X) \cap \mathcal{BF}(Y)| = \mathcal{BF}(X) *_{\text{bitwise}} \mathcal{BF}(Y)$$

For static datasets, this method has real advantages, including speed, scalability, and performance. Limitations are based on a variety of features and on the ability to change the set of possible items. Later we will discuss *locally sensitive hashing*, which further iterates on lookup speed with lower risks of collision in high-dimensional spaces, and some similar ideas will reemerge.

Feature Stores

So far, we have focused on recommendation systems that we might call *pure collaborative filtering*. We've made use of the user- or item-similarity data only when attempting to make good recommendations. If you've been wondering, "Hey, what about information about the actual users and items?" your curiosity will now be sated.

There are a huge variety of reasons you could be interested in features in addition to your previous CF methods. Let's list a few high-level concerns:

- You may wish to show new users a specific set of items first.
- You may wish to consider geographic boundaries in your recommendations.
- Distinguishing between children and adults may be important for the types of recommendations they're given.
- Item features may be used to ensure high-level diversity in the recommendations (more to come in [Chapter 15](#)).
- User features can enable various kinds of experimental testing.

- Item features could be used to group items into sets for contextual recommendations (more to come in [Chapter 15](#)).

In addition to these issues, another kind of feature is often essential: real-time features. While the point of feature stores is to provide real-time access to all the necessary features, it's worthwhile to distinguish stable features that change infrequently from real-time features that we anticipate will change often.

Some important examples of a real-time feature store are dynamic prices, current item availability, *trending* status, wish-list status, and so on. These features may change throughout the day, and we want their values in the feature store to be mutable in real-time via other services and systems. Therefore, the real-time feature store will need to provide API access for feature mutation. This is something you may not want to provide for *stable* features.

When we design our feature store, we're likely to want the stable features to be built from data warehouse tables via ETLs and transformations, and we likely want the real-time features to be built this way as well, but on a faster schedule or allowing API access for mutation. In either case, the key quality of a feature store is *very fast read access*. It's often a good idea to separately build feature stores for offline training of models that can be built in test to ensure support for new models.

So how might the architecture and implementation look? See [Figure 6-3](#).

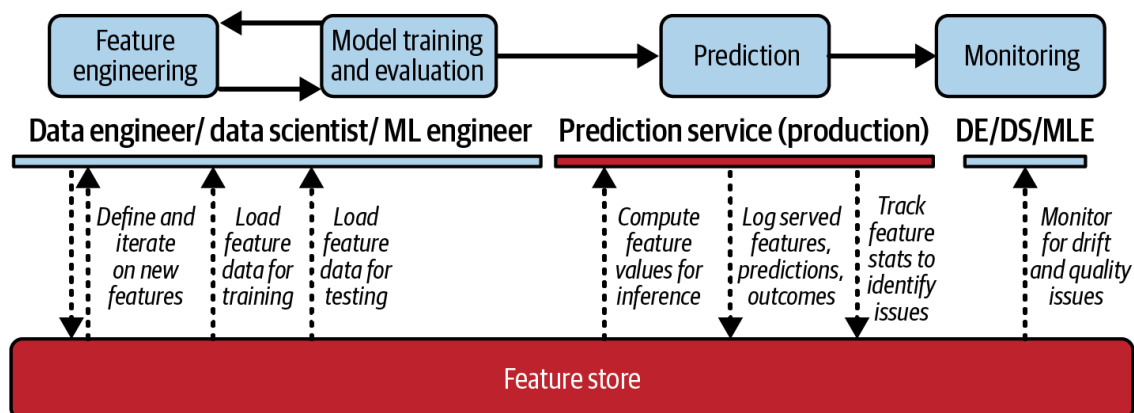


Figure 6-3. Demonstration of a feature store

Designing a feature store involves designing pipelines that define and *transform the features into that store* (coordinated via things like Airflow, Luigi, Argo, etc.) and often look similar to the type of data pipelines used in building our collector. One additional complication that the feature store needs to concern itself with is a speed layer. During our discussion of the lambda architecture earlier in this chapter, we mentioned that we can think of batch data processing for the collector and a more rapid speed layer for intermediary updates, but this is even more important for the feature store. The feature store may also need a *streaming layer*. This layer operates on continuous streams of data and can perform data transformations on those; it then writes the appropriate output to the online feature store in real time. This adds complexity because data transformations on streaming data present a very different set of challenges and often require different algorithmic strategies. Some technologies that help here are Spark Streaming and Kinesis. You'll also need to configure the system to properly handle the data stream, the most common of which is Kafka. Data streaming layers involve many components and architectural considerations that fall outside our scope; if you're considering getting started with Kafka, check out [*Kafka: The Definitive Guide*](#) by Gwen Shapira et al. (O'Reilly).

A feature store also needs a *storage layer*; many approaches exist here, but using a NoSQL database is common, especially in the online feature store. The reason is faster retrieval and the nature of the data storage. Feature stores for recommendation systems tend to be very key based (i.e., *get the features for this user*, or *get the features for this item*), which lend themselves well to key-value stores. Some example technologies here are DynamoDB, Redis, and Cassandra. The storage layer for an off-line feature store may simply be an SQL-style database to reduce complexity, but instead you'll pay a tax of a delta between offline and online. This delta and others like it are called [*training-serving skew*](#).

A unique but essential aspect of feature stores is the *registry*. A registry is incredibly useful for a feature store because it coordinates existing features and information on how they're defined. A more sophisticated instance of a registry also includes input and output schemas with typing, and distributional expectations. These are contracts that the data pipe-

lines must adhere to and satisfy to avoid populating your feature store with garbage data. Additionally, the registry's definitions allow parallel data scientists and ML engineers to develop new features, use one another's features, and generally understand the assumptions of features their models may utilize.

One important advantage of these registries is that they incentivize alignment between teams and developers. In particular, if you decide you care about *country* for your user, and you see a feature *country* in the registry, you're more likely to use that (or ask the developer who's assigned to this feature in the registry) than to make a new one from scratch. Practically, data scientists make hundreds of small decisions and assumptions when defining their models, and this removes some of that load that's relying on the existing resources.

MODEL REGISTRIES

A closely related concept to feature registries is model registries. The concepts have a lot in common, but we caution you to think of them differently. A great model registry can have type contracts for the input and output of your models, and can serve many of the same benefits around alignment and clarity. A feature registry should really be focused on definitions of the business logic and features. Because feature engineering can also be model driven, speaking clearly about the differences between these two things can be challenging, so to sum it up, we'll focus on what they serve: a model registry concerns itself with ML models and the relevant metadata, whereas a feature registry concerns itself with features that models will use.

Finally, we need to talk about *serving* these features. Backed by the appropriately performant storage layer, we need to serve via API request the necessary feature vectors. Those feature vectors are details about the user that the model will need when serving recommendations—for example, the user's location or content age restrictions. The API can serve back the entire set of features for the key or allow for more specification. Often the responses are JSON serialized for fast data transfer. It's important that the features being served are the *most up-to-date set of features*, and la-

tency here is expected to be < 100 ms for more serious industrial applications.

One important caveat here is that for offline training, these feature stores need to accommodate *time travel*. Because our goal during training is to give the model the appropriate data to learn in the *most generalizable way*, when training our model, it's crucial to not give it access to features out of time. This is called *data leakage* and can cause massive divergence in performance between training and production. The feature store for offline training thus must have knowledge of the features through time, so that during training, a time index may be provided to get the features as they were then. These `as_of` keys can be tied to the historical training data as we *replay* the history of what the user-item interactions looked like.

With these pieces in place—and the important monitoring this system needs—you'll be able to serve offline and online features to your models. In [Part III](#), you will see model architectures that make use of them.

You're likely familiar with the concept of leakage based on what you know about ML: corrupted performance metrics result because the training of the model had access to data that was supposed to be reserved for model performance evaluation.

Data leakage in ML is divided into *feature leakage* and *training example leakage*. For recommendation systems, data leakage has the additional challenge of temporal leakage, or nonstationarity leakage. The real danger is that in recommendation systems, we see the same observational unit, a user, over and over, and observe a datum each time we see them. When we see them, other aspects of the system may have changed, and in reality we want to use features in our model that are the most up to date as of that observation. Both in features and training examples, to avoid leakage we need to always be thinking of our system's timeline. This is why data preparation for recommendation systems is inherently time dependent. You will see in [Chapter 11](#) that our accuracy metrics will need to explicitly consider train-test splitting with respect to this time axis, and then be further grouped by the user. This also means that training recommendation systems often requires more resources than many other task types.

Summary

We've discussed not only the crucial components necessary to hydrate your system and serve recommendations, but also some of the engineering building blocks needed to make those components a reality. Equipped with data loaders, embeddings, feature stores, and retrieval mechanisms, we are ready to start constructing our pipeline and system topology.

In the next chapter, we'll focus our sights on MLOps and the rest of the engineering work required to build and iterate on these systems. It's going to be important for us to think carefully about deployment and monitoring so our recommendation systems are constrained to life in IPython Notebooks.

Continue onward to see the architectural considerations to move to production.